

Assingment—8

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Prompt: generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Requirements:

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Code:

```
import re
```

```
def is_strong_password(password):
```

```
    """
```

```
    Validates if a password meets the following criteria:
```

- At least 8 characters long.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one digit.
- Contains at least one special character (non-alphanumeric and not space).
- Does not contain any spaces.

```
    """
```

```
    if len(password) < 8:
```

```
        return False
```

```

if ' ' in password:
    return False

if not re.search(r'[A-Z]', password):
    return False

if not re.search(r'[a-z]', password):
    return False

if not re.search(r'\d', password):
    return False

if not re.search(r'^A-Za-z0-9\s$', password):
    return False

return True

#example usage

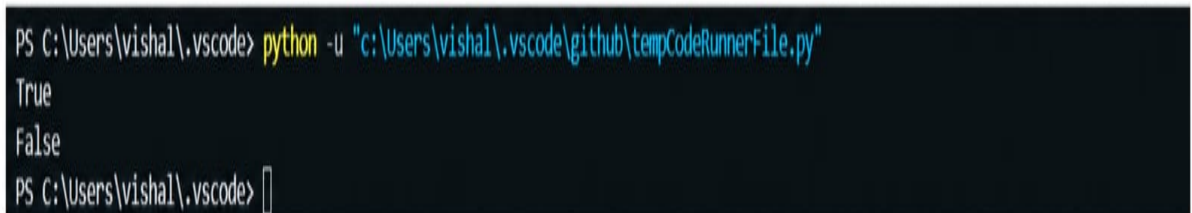
print(is_strong_password("StrongPass1!")) # True
print(is_strong_password("weakpass")) # False

#Example Assert Test Cases:

assert is_strong_password("&quot;Abcd@123&quot;") == True
assert is_strong_password("&quot;abcd123&quot;") == False
assert is_strong_password("&quot;ABCD@1234&quot;") == True

```

output:



```

PS C:\Users\visal\.vscode> python -u "c:\Users\visal\.vscode\github\tempCodeRunnerFile.py"
True
False
PS C:\Users\visal\.vscode> 

```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Prompt:

generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Requirements:

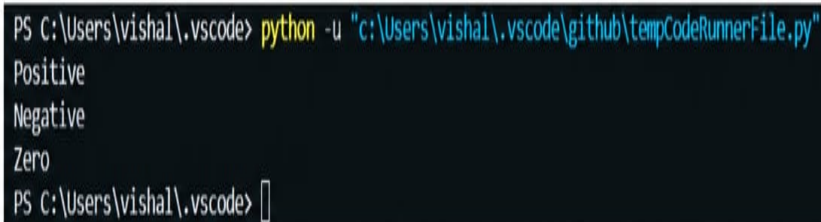
- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Code:

```
def classify_number(n):  
    """  
    Classifies a number as Positive, Negative, or Zero.  
    Handles invalid inputs like strings and None.  
    """  
    if n is None or isinstance(n, str):  
        return "Invalid input"  
  
    if n > 0:  
        return "Positive"  
    elif n < 0:  
        return "Negative"  
    else:  
        return "Zero"  
  
#example usage  
print(classify_number(5)) # Positive  
print(classify_number(-3)) # Negative  
print(classify_number(0)) # Zero  
  
#Example Assert Test Cases:
```

```
assert classify_number(10) == "Positive";
assert classify_number(-5) == "Negative";
assert classify_number(0) == "Zero";
```

output:



```
PS C:\Users\vishal\.vscode> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
Positive
Negative
Zero
PS C:\Users\vishal\.vscode> 
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Prompt: generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases (empty strings, identical words).

Code:

Import re

```
def is_anagram(str1, str2):
```

```
    """
```

```
    Checks if two strings are anagrams of each other.
```

```
    Ignores case, spaces, and punctuation.
```

```
    """
```

```
    # Normalize the strings: remove spaces and punctuation, convert to lowercase
```

```
    str1_normalized = re.sub(r'[\W_]', '', str1).lower()
```

```
    str2_normalized = re.sub(r'[\W_]', '', str2).lower()
```

```

# Check if the sorted characters of both strings are the same
return sorted(str1_normalized) == sorted(str2_normalized)

#example usage

print(is_anagram("Listen", "Silent")) # True
print(is_anagram("Hello", "World")) # False
print(is_anagram("", "")) # True

#Example Assert Test Cases:

assert is_anagram(&quot;listen&quot;, &quot;silent&quot;) == True
assert is_anagram(&quot;hello&quot;, &quot;world&quot;) == False
assert is_anagram(&quot;Dormitory&quot;, &quot;Dirty Room&quot;) == True

```

output:

```

PS C:\Users\vishal\.vscode> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
True
False
True
PS C:\Users\vishal\.vscode> 

```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

Task: Ask AI to generate at least 3 assert-based tests for an

Inventory class with stock management.

prompt: generate at least 3 assert-based tests for an

Inventory class with stock management.

Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

code:

```
class Inventory:
```

```
    def __init__(self):
```

```
        self.stock = {}
```

```
    def add_item(self, name, quantity):
```

```
        if not isinstance(quantity, int) or quantity <= 0:
```

```
            raise ValueError("Quantity must be a positive integer")
```

```
        self.stock[name] = self.stock.get(name, 0) + quantity
```

```
    def remove_item(self, name, quantity):
```

```
        if not isinstance(quantity, int) or quantity <= 0:
```

```
            raise ValueError("Quantity to remove must be a positive integer")
```

```
        if name not in self.stock:
```

```
            raise ValueError(f"Item '{name}' not in inventory")
```

```
        if self.stock[name] < quantity:
```

```
            # Optionally, you could raise an error here or set to 0
```

```
            # For this example, let's assume we remove what's available and stock becomes 0
```

```
            # Or, as per example, simply decrement
```

```
            self.stock[name] = max(0, self.stock[name] - quantity)
```

```
        else:
```

```
            self.stock[name] -= quantity
```

```
    def get_stock(self, name):
```

```
        return self.stock.get(name, 0)
```

```
#example usage
```

```
inventory = Inventory()
```

```
inventory.add_item("apple", 10)
```

```
print(inventory.get_stock("apple")) # 10

#example test cases

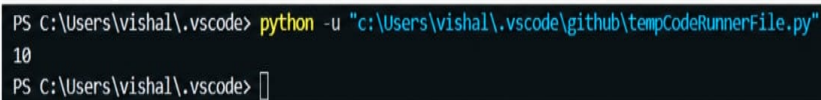
inventory.add_item("banana", 5)

assert inventory.get_stock("banana") == 5 # Added banana

inventory.remove_item("banana", 3)

assert inventory.get_stock("banana") == 2 # Removed 3 bananas
```

output:



```
PS C:\Users\vishal\.vscode> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
10
PS C:\Users\vishal\.vscode> 
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Prompt:

generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Requirements:

- o Validate “MM/DD/YYYY” format.
- o Handle invalid dates.
- o Convert valid dates to “YYYY-MM-DD”

code:

```
import datetime
```

```
def validate_and_format_date(date_str):
```

```
    """
```

```
    Validates if a date string is in 'MM/DD/YYYY' format and converts it to 'YYYY-MM-DD'.
```

Handles invalid dates by returning 'Invalid Date'.

```
"""
```

```
try:
```

```
    # Attempt to parse the date string in MM/DD/YYYY format
```

```
    dt_obj = datetime.datetime.strptime(date_str, "%m/%d/%Y")
```

```
    # If successful, format it to YYYY-MM-DD
```

```
    return dt_obj.strftime("%Y-%m-%d")
```

```
except ValueError:
```

```
    # If parsing fails (invalid format or invalid date like 02/30/YYYY)
```

```
    return "Invalid Date"
```

```
#example usage
```

```
print(validate_and_format_date("12/31/2020")) # 2020-12-31
```

```
print(validate_and_format_date("02/30/2020")) # Invalid Date
```

```
#example test cases
```

```
assert validate_and_format_date("12/31/2020") == "2020-12-31" # Valid date
```

```
assert validate_and_format_date("02/30/2020") == "Invalid Date" # Invalid date
```

```
assert validate_and_format_date("13/01/2020") == "Invalid Date" # Invalid month
```

output:

```
PS C:\Users\vishal\.vscode> python -u "c:\Users\vishal\.vscode\github\tempCodeRunnerFile.py"
2020-12-31
Invalid Date
PS C:\Users\vishal\.vscode> 
```